

클래스

1. 클래스

2. module

3. package

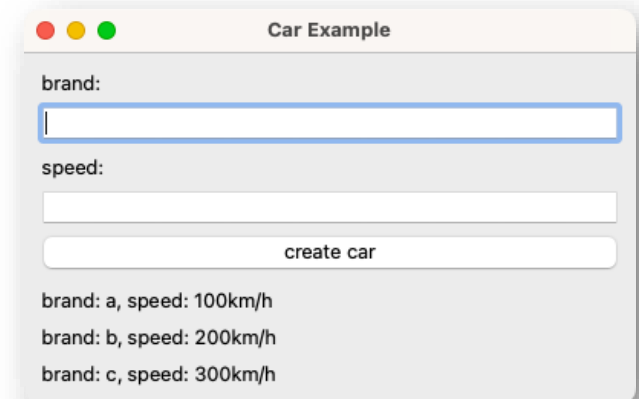
■ 클래스

- 객체지향의 가장 기본적 개념
- 관련된 속성(모습)과 동작(기능)을 하나의 범주로 묶어 실세계의 사물(객체) 표현
 - 객체를 생성하기 위한 설계도 (템플릿) 역할
- 클래스를 사용하는 이유
 1. 새로운 자료형
 2. 기능의 집합
 3. 다형성 구현을 위한 상속

ex) 자동차 공장 - [class_car_exam.py 소스코드](#)

- 1) 자동차 설계 : 자동차 클래스 정의
- 2) 자동차 기능 : 자동차 클래스의 메소드 정의
- 3) 자동차 특징 : 자동차 클래스의 변수 정의
- 4) 자동차 생산 : 인스턴스 생성

* 기능과 특징이 다른 여러 종류의 자동차 생산 : 자동차 클래스 상속



Car Example

brand:

speed:

brand: a, speed: 100km/h
brand: b, speed: 200km/h
brand: c, speed: 300km/h

■ 클래스

● 클래스의 기본 형태

● 객체 생성 후 사용 → 인스턴스 메소드

● 객체 생성 없이 사용

1) 정적 메소드

→ 단순히 기능(유틸)만 사용

2) 클래스 메소드

→ 클래스 변수 접근용

```
전역변수 = 500
```

```
class 클래스:
    클래스변수 = 100

    def __init__(self): # 생성자
        self.인스턴스변수 = 10

    def 인스턴스메소드(self):
        self.인스턴스변수 += 10

    @staticmethod
    def 정적메소드():
        지역변수 = 100
        print(f'함수 동작 {지역변수}')

    @classmethod
    def 클래스메소드(cls):
        print(cls.클래스변수)
```

```
객체1 = 클래스()
객체1.인스턴스메소드()
```

```
객체2 = 클래스()
객체2.인스턴스메소드()
```

```
클래스.정적메소드()
클래스.클래스메소드()
```

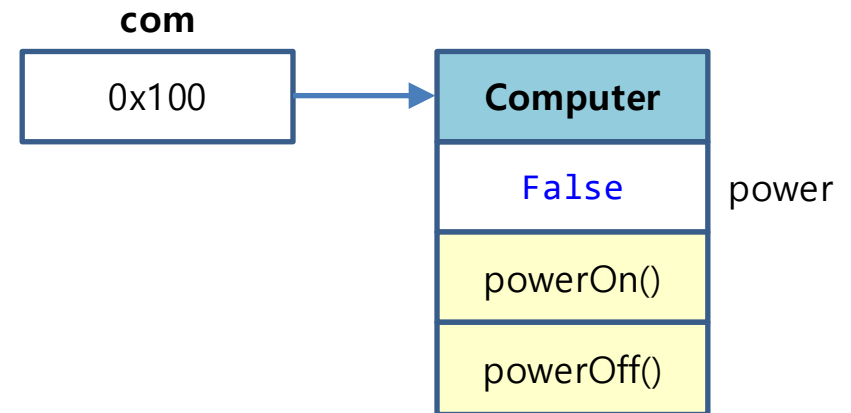
■ 클래스

- 클래스(Class) : 설계도면
- 객체(Object) : 클래스(설계도면)에 의해 만들어진 완성물(제품)
- 인스턴스(Instance) : 객체와 같은 의미
 - com은 객체, Computer 클래스의 인스턴스

```
class Computer:
    def powerOn(self):
        self.power = True
        print('전원 ON')

    def powerOff(self):
        self.power = False
        print('전원 OFF')

com = Computer()
com.powerOn()
com.powerOff()
```



■ 클래스

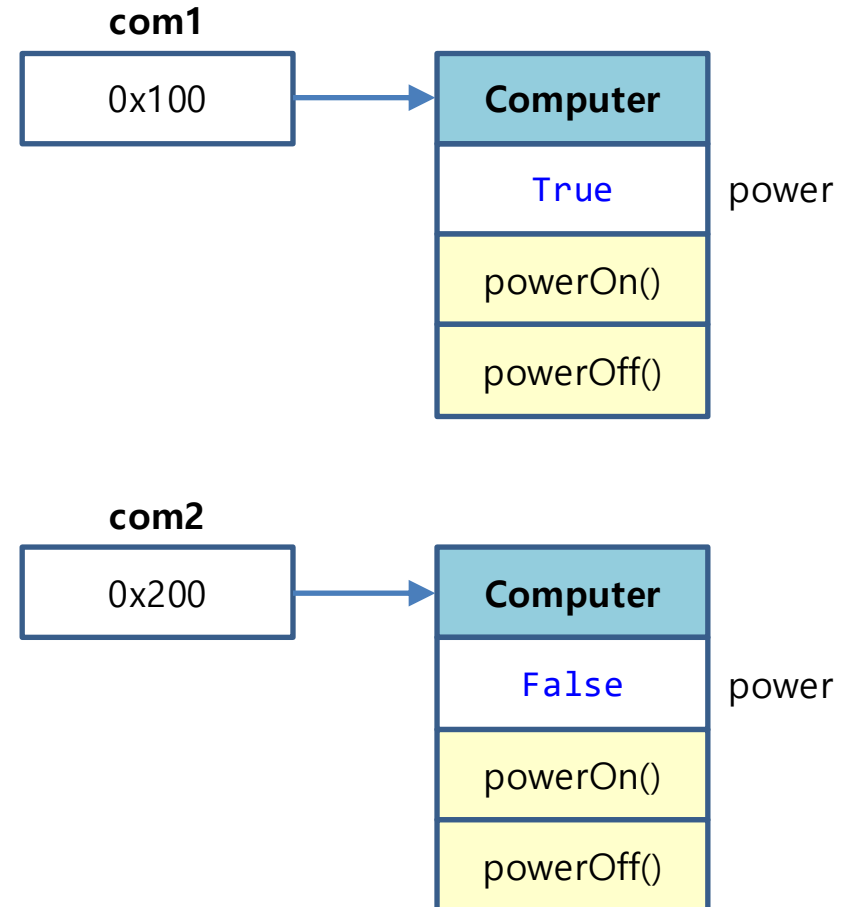
● 인스턴스(Instance) 생성

```
class Computer:
    def powerOn(self):
        self.power = True
        print('전원 ON')

    def powerOff(self):
        self.power = False
        print('전원 OFF')

com1 = Computer()
com1.powerOn()

com2 = Computer()
com2.powerOff()
```



■ 클래스

● 학생 정보를 저장할 수 있는 클래스

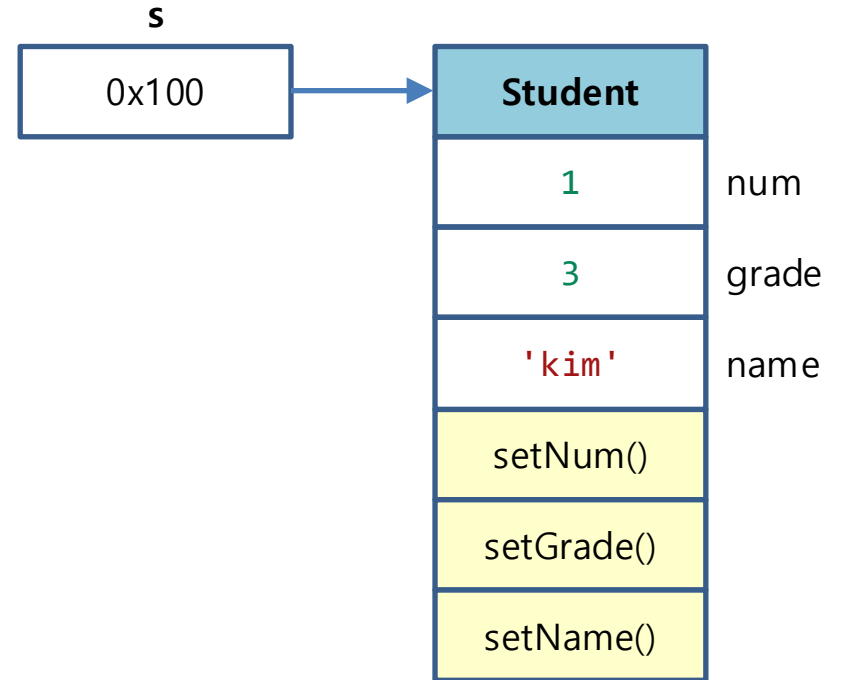
```
class Student:
    def setNum(self, num):
        self.num = num

    def setGrade(self, grade):
        self.grade = grade

    def setName(self, name):
        self.name = name

    def __str__(self):
        return f'학번: {self.num}, 학년: {self.grade}, 이름: {self.name}'

s = Student()
s.setNum(1)
s.setGrade(3)
s.setName('kim')
print(s)
```



■ 연습문제

- 아래의 코드가 실행되어 결과와 같이 출력될 수 있도록 메소드 작성하기

```
class Book:

    # 코드 작성

book1 = Book()
book1.setTitle('Python')
book1.setPrice(10000)
print(book1.title, book1.price)

book2 = Book()
book2.setTitle('Java')
book2.setPrice(10000)
print(book2.title, book2.price)
```

Python 10000

Java 10000

■ 클래스

● 생성자를 이용한 초기화

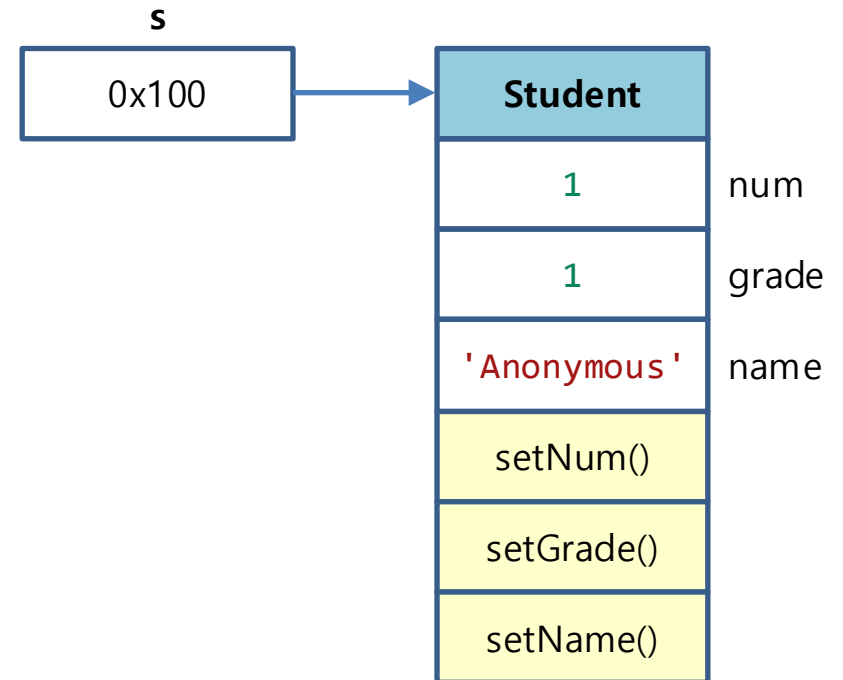
```
class Student:
    def __init__(self):
        self.num = 1
        self.grade = 1
        self.name = 'Anonymous'

    def setNum (self, num):
        self.num = num

    def setGrade (self, grade):
        self.grade = grade

    def setName (self, name):
        self.name = name

s = Student()
print(s.num, s.grade, s.name)
```



■ 클래스

● 생성자를 이용한 초기화

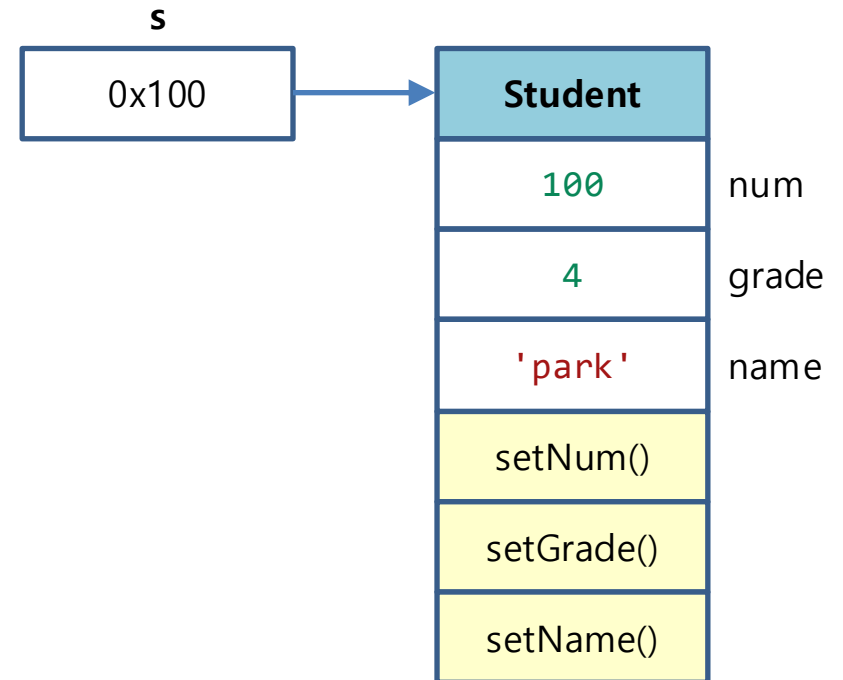
```
class Student:
    def __init__(
        self, num, grade, name):
        self.num = num
        self.grade = grade
        self.name = name

    def setNum (self, num):
        self.num = num

    def setGrade (self, grade):
        self.grade = grade

    def setName (self, name):
        self.name = name

s = Student(100, 4, 'park')
print(s.num, s.grade, s.name)
```



■ 연습문제

- 아래의 코드가 실행되어 결과와 같이 출력될 수 있도록 생성자 작성하기

```
class Product:

    # 생성자 작성

    def show_info(self):
        return 'name=%s, code=%s, count=%s' % (
            self.name, self.code, self.count)

p1 = Product('clock', 'c0001', 500)
print(p1.show_info())

p2 = Product('tv', 't0001', 1500)
print(p2.show_info())
```

```
name=clock, code=c0001, count=500
name=tv, code=t0001, count=1500
```

■ 연습문제

- 내용을 저장하고 저장된 내용을 읽을 수 있는 Letter 클래스 작성하기

```
class Letter:
    # 생성자 작성, 변수 초기화

    # write 메소드 작성

    # show 메소드 작성

letter1 = Letter()
letter1.write('가나다라')
letter1.write('카타파하')
print(letter1.show())

letter2 = Letter()
letter2.write('ABCD')
letter2.write('WXYZ')
print(letter2.show())
```

가나다라카타파하
ABCDWXYZ

■ 연습문제

- 자동차 옵션을 추가하고 추가된 옵션을 확인할 수 있는 Car 클래스 작성하기

```
class Car:
    # 생성자 작성, 변수 초기화

    # add_option 메소드 작성

    # show_option 메소드 작성

car1 = Car()
car2 = Car()

car1.add_option('전동 트렁크')
car1.add_option('통풍 시트')
print(car1.show_option())

car2.add_option('뒷자리 에어백')
car2.add_option('하이패스')
print(car2.show_option())
```

전동 트렁크, 통풍 시트
뒷자리 에어백, 하이패스

■ 클래스

● 상속

- 부모 클래스

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return '%s makes a noise' % self.name
```

- 자식 클래스

```
class Dog(Animal):
    pass
```

- 사용

```
dog = Dog('ShuShu')
print(dog.speak())
```

```
ShuShu makes a noise
```

■ 클래스

● 기능 추가

- 부모 클래스에는 없고 자식 클래스만 가지는 메소드 추가

```
class Dog(Animal):  
    def run(self):  
        return '%s run' % self.name  
  
dog = Dog('ShuShu')  
print(dog.speak())  
print(dog.run())
```

```
ShuShu makes a noise  
ShuShu run
```

■ 클래스

● 오버라이드

- 부모 클래스의 메소드를 재정의하여 자식 클래스만의 기능으로 작성

```
class Dog(Animal):  
  
    def speak(self):  
        return '%s 멍멍' % self.name  
  
    def run(self):  
        return '%s run' % self.name  
  
dog = Dog('ShuShu')  
print(dog.speak())  
print(dog.run())
```

ShuShu 멍멍

ShuShu run

■ 클래스

● PyQt 상속 예)

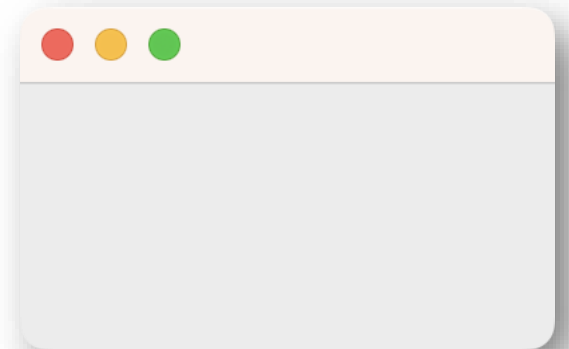
- 기본 Window 사용

```
import sys
from PyQt5.QtWidgets import *

app = QApplication.instance()
if not app:
    app = QApplication(sys.argv)

mywindow = QMainWindow()
mywindow.show()

app.exec_()
```



■ 클래스

● PyQt 상속 예)

- Window를 상속받은 후 커스터마이징 - 종료 버튼 추가

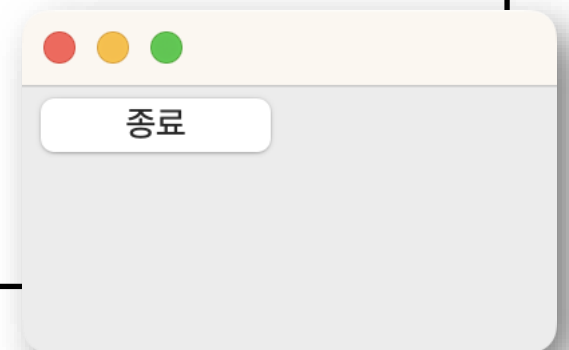
```
import sys
from PyQt5.QtWidgets import *

class MyWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.btn = QPushButton("종료", self)

app = QApplication.instance()
if not app:
    app = QApplication(sys.argv)

mywindow = MyWindow()
mywindow.show()

app.exec_()
```

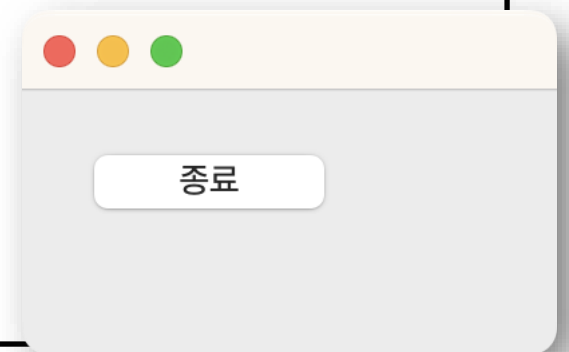


■ 클래스

● PyQt 상속 예)

- Window를 상속받은 후 커스터마이징 - 종료 버튼 위치 지정, 종료 기능 추가

```
...  
  
class MyWindow(QMainWindow):  
    def __init__(self):  
        super().__init__()  
        self.btn = QPushButton("종료", self)  
        self.btn.move(20, 20)  
        self.btn.clicked.connect(self.btn_clicked)  
  
    def btn_clicked(self):  
        self.close()  
  
app = QApplication.instance()  
if not app:  
    app = QApplication(sys.argv)  
  
...
```



■ 연습문제

- Shape 클래스를 상속받는 Rectangle, Triangle 클래스 작성하기
 - draw(), getArea() 오버라이드

```
class Shape:
    def __init__(self, width, height, color):
        self.width = width
        self.height = height
        self.color = color

    def draw(self):
        print('draw')

    def getArea(self):
        print(self.width * self.height)
```

코드 작성 - Rectangle, Triangle 클래스

```
rect = Rectangle(20, 20, 'blue')
rect.draw()
rect.getArea()

tri = Triangle(20, 20, 'red')
tri.draw()
tri.getArea()
```

```
Draw Rectangle
area => 400
Draw Triangle
area => 200
```

■ 모듈 (module)

- 파이썬 코드를 논리적으로 묶어서 관리하고 사용할 수 있도록 만들어 둔 것
- 보통 py 확장자를 가지는 1개의 파이썬 파일이 하나의 모듈이 됨
- 함수, 클래스, 변수를 정의할 수 있으며, 실행 코드도 포함 가능
- 기본적으로 많은 표준 라이브러리 모듈을 제공하고 있으며,
필요에 따라서 외부 라이브러리 모듈을 설치하여 사용하는 것도 가능
- import문을 사용하여 1개 이상의 모듈을 불러들임
- 기본 사용
 - import module1, module2, ... , moduleN

```
import random
num = random.randint(1, 10)
print(num)
```

■ 모듈 (module)

● 모듈 작성 및 사용 (import)

calculator.py

```
def plus(first, second):  
    return first + second  
  
def minus(first, second):  
    return first - second  
  
def multiply(first, second):  
    return first * second  
  
def divide(first, second):  
    return first / second
```

import calculator

```
result1 = calculator.plus(5, 3)  
print(result1)
```

```
result2 = calculator.minus(5, 3)  
print(result2)
```

```
result3 = calculator.multiply(5, 3)  
print(result3)
```

```
result4 = calculator.divide(5, 3)  
print(result4)
```

8

2

15

1.6666666666666667

■ 모듈 (module)

● 모듈 작성 및 사용 (from - import)

calculator.py

```
def plus(first, second):  
    return first + second  
  
def minus(first, second):  
    return first - second  
  
def multiply(first, second):  
    return first * second  
  
def divide(first, second):  
    return first / second
```

```
from calculator \  
    import plus, minus, multiply, divide
```

```
result1 = plus(5, 3)  
print(result1)
```

```
result2 = minus(5, 3)  
print(result2)
```

```
result3 = multiply(5, 3)  
print(result3)
```

```
result4 = divide(5, 3)  
print(result4)
```

■ 모듈 (module)

● 모듈 작성 및 사용 (alias)

```
from calculator \
    import plus as p, minus as m, \
        multiply as mul, divide as div

result1 = p(5, 3)
print(result1)

result2 = m(5, 3)
print(result2)

result3 = mul(5, 3)
print(result3)

result4 = div(5, 3)
print(result4)
```

```
import calculator as cal

result1 = cal.plus(5, 3)
print(result1)

result2 = cal.minus(5, 3)
print(result2)

result3 = cal.multiply(5, 3)
print(result3)

result4 = cal.divide(5, 3)
print(result4)
```

■ 모듈 (module)

- import 될 때 작성된 코드가 실행되므로 현재 실행상태가 main의 역할인지 모듈로써 실행되는지 판단하여 처리
- `__name__` 속성
 - main 실행 : `'__main__'` 문자열 반환
 - module로 import : `'파일명'(모듈명)` 문자열 반환

```
def plus(first, second):  
    return first + second  
  
def minus(first, second):  
    return first - second  
  
def multiply(first, second):  
    return first * second  
  
def divide(first, second):  
    return first / second  
  
print(__name__)  
print(plus(10, 20))
```

메인으로 사용

```
__main__  
30
```

모듈로 사용

```
calculator  
30
```

■ 모듈 (module)

● `__name__` 속성을 이용한 코드 처리

calculator.py

```
def plus(first, second):  
    return first + second  
  
def minus(first, second):  
    return first - second  
  
def multiply(first, second):  
    return first * second  
  
def divide(first, second):  
    return first / second  
  
if __name__ == '__main__':  
    print(__name__)  
    print(plus(10, 20))
```

■ 연습문제

- 결과와 같이 동작될 수 있도록 mymod 모듈 작성하고 사용하기

mymod.py

```
# myfunc 함수 작성 - 두개의 수를 인자로 받은 후 빼기 연산결과 반환
```

```
# myfunc 모듈 가져오기
```

```
result = myfunc(1, 2)  
print('myfunc 실행결과 %s' % result)
```

결과

```
myfunc 실행결과 -1
```

■ 연습문제

- 결과와 같이 동작될 수 있도록 pw_encoder 모듈 작성하고 사용하기

pw_encoder.py

```
# get_pw 함수 작성

# 암호화된 비밀번호 생성 코드
# import hashlib
# pw = '1234'
# pw_to_bytes = bytes(pw, 'utf8')
# m = hashlib.sha256()
# m.update( pw_to_bytes )
# print( m.hexdigest() )
```

```
# pw_encoder 모듈 가져오기

raw_password = '1234'
encoded_password = get_pw(raw_password)
print(f'{raw_password} => {encoded_password}')
```

결과

```
1234 => 03ac674216f3e15c761ee1a5e255f067953623c8b388b4459e13f978d7c846f4
```

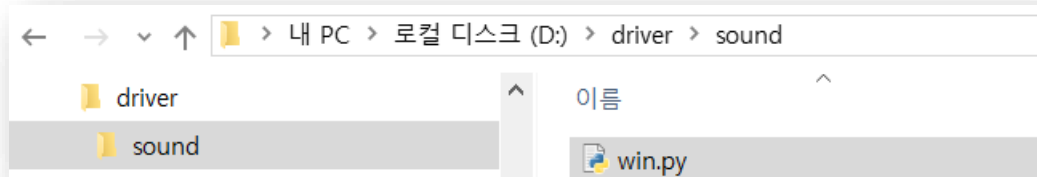
■ 패키지 (Package)

● 모듈을 계층(디렉토리) 구조로 관리하는 것

ex) 모듈명이 driver.sound.win 인 경우

패키지 : driver, sound

모듈 : win



● __init__.py

- 패키지 수준의 공통 변수를 설정하거나 모듈을 미리 불러오는 초기화 작업

■ 패키지 (Package)

● driver 패키지 하위 모듈 import

패키지 구조

```
driver/  
  sound/  
    mp3.py  
    wav.py  
  vga/  
    ati.py  
    nvidia.py
```

driver/sound/mp3.py

```
print('sound mp3!')  
  
def sound():  
    print('mp3~~')
```

driver/sound/wav.py

```
print('sound wav!')  
  
def sound():  
    print('wav~~')
```

driver/vga/ati.py

```
print('vga ati!')
```

driver/vga/nvidia.py

```
print('vga nvidia!')
```

모듈 사용

```
import driver.sound.mp3  
from driver.sound.wav import *  
import driver.vga.ati  
from driver.vga.nvidia import *  
  
print('----')  
  
driver.sound.mp3.sound()  
sound() # driver.sound.wav
```

```
sound mp3!  
sound wav!  
vga ati!  
vga nvidia!  
----  
mp3~~  
wav~~
```

■ 패키지 (Package)

● __init__.py 사용

패키지 구조

```
driver/  
  sound/  
    mp3.py  
    wav.py  
  vga/  
    ati.py  
    nvidia.py
```

driver/sound/mp3.py

```
print('sound mp3!')  
  
def sound():  
    print('mp3~~')
```

driver/sound/wav.py

```
print('sound wav!')  
  
def sound():  
    print('wav~~')
```

모듈 사용

```
from driver.sound import *  
  
mp3.sound() # Error  
wav.sound()
```

```
mp3.sound()  
^^^
```

NameError: name 'mp3' is not defined

■ 패키지 (Package)

● __init__.py 사용 (모듈 미리 불러오기)

패키지 구조

```
driver/  
  sound/  
    mp3.py  
    wav.py  
    __init__.py  
  vga/  
    ati.py  
    nvidia.py
```

driver/sound/mp3.py

```
print('sound mp3!')  
  
def sound():  
    print('mp3~~')
```

driver/sound/wav.py

```
print('sound wav!')  
  
def sound():  
    print('wav~~')
```

driver/sound/__init__.py

```
__all__ = ['mp3', 'wav']
```

모듈 사용

```
from driver.sound import *
```

```
mp3.sound()  
wav.sound()
```

```
sound mp3!  
sound wav!  
mp3~~  
wav~~
```

■ 패키지 (Package)

● __init__.py 사용 (공통 변수 설정)

패키지 구조

```
driver/  
  sound/  
    mp3.py  
    wav.py  
    __init__.py  
  vga/  
    ati.py  
    nvidia.py
```

driver/sound/__init__.py

```
SOUND_VERSION = "v2.1"  
DEFAULT_VOLUME = 80  
  
__all__ = ['mp3', 'wav']
```

driver/sound/mp3.py

```
from . import SOUND_VERSION, DEFAULT_VOLUME  
  
print('sound mp3!')  
  
def sound():  
    print('mp3~~')  
    print({'version': SOUND_VERSION})  
    print({'volume': DEFAULT_VOLUME})
```